

Laboratorio 5 - Parte 2

Cross-Site Request Forgery (CSRF)

Programacion Web - Backend | FUNDATEC D.A.S | Jonathan Naranjo

1. Que es un ataque CSRF y como se ejecuta?

Un ataque de **Cross-Site Request Forgery (CSRF)** es una vulnerabilidad que explota la confianza que un servidor web deposita en el navegador de un usuario autenticado. El atacante engaña al navegador de la victima para que envíe, sin su conocimiento, una petición HTTP legítima a un servidor en el que ya tiene una sesión activa. Dado que los navegadores incluyen automáticamente las cookies de sesión en cada petición al dominio correspondiente, el servidor recibe la solicitud como si la hubiera generado el propio usuario y la procesa con sus privilegios.

Para que el ataque se produzca deben cumplirse tres condiciones simultáneamente: **(1)** el usuario debe estar autenticado en la aplicación objetivo y tener una cookie de sesión activa en su navegador; **(2)** la petición que el atacante desea falsificar debe ser predecible, es decir, el atacante debe conocer de antemano la URL, el método HTTP y los parámetros que componen la acción (por ejemplo, `POST /api/transfer?amount=500&to=attacker`); y **(3)** el servidor no debe implementar mecanismos de verificación de origen adicionales más allá de la cookie de sesión.

2. Flujo del ataque desde el navegador del usuario

El flujo típico de un ataque CSRF se desarrolla en los siguientes pasos:

- **Paso 1 - Autenticación legítima:** La víctima inicia sesión en la aplicación objetivo (por ejemplo, su banco en línea). El servidor emite una cookie de sesión que queda almacenada en el navegador.
- **Paso 2 - Visita al sitio malicioso:** Mientras mantiene la sesión abierta, la víctima visita una página web controlada por el atacante, generalmente a través de un enlace en un correo electrónico o un mensaje de chat.
- **Paso 3 - Disparo de la petición falsificada:** La página maliciosa contiene código HTML o JavaScript que, de forma automática y silenciosa, genera una petición HTTP hacia el servidor de la aplicación objetivo (por ejemplo, un formulario oculto con `action='https://banco.com/transferir'` que se envía al cargar la página).

- **Paso 4 - El navegador adjunta la cookie:** El navegador incluye automaticamente la cookie de sesion de la victima en la peticion, ya que esta va dirigida al dominio legitimo.

- **Paso 5 - El servidor ejecuta la accion:** El servidor recibe la peticion con una cookie valida, la considera autorizada y ejecuta la operacion sin que la victima lo haya solicitado.

El impacto para el usuario final puede ser severo: perdida de datos, transferencias economicas no autorizadas, cambios de credenciales o cualquier otra accion que el usuario este autorizado a realizar en la aplicacion. El ataque es especialmente peligroso porque la victima no necesita ejecutar ningun archivo; basta con visitar la pagina maliciosa.

3. Mitigacion mediante tokens anti-CSRF

La contramedida mas efectiva y ampliamente adoptada es el uso de **tokens anti-CSRF** (tambien llamados *synchronizer tokens*). Al momento de generar un formulario o iniciar una sesion, el servidor crea un valor aleatorio, impredecible y unico (el token CSRF), lo almacena en la sesion del usuario del lado del servidor y lo incrusta en la respuesta, generalmente como campo oculto en formularios HTML o como encabezado HTTP personalizado en peticiones AJAX. En cada solicitud que modifica estado (POST, PUT, PATCH, DELETE), el cliente debe enviar ese token de vuelta, y el servidor verifica que el token recibido coincida con el almacenado en la sesion antes de procesar la peticion.

Un atacante que controla un sitio de origen diferente no puede leer el token, ya que la **Same-Origin Policy** del navegador impide que JavaScript de un dominio cruzado acceda al contenido de las respuestas de otro dominio. Sin el token correcto, el servidor rechaza la peticion falsificada con un error *403 Forbidden*. Esta tecnica se complementa con el atributo **SameSite=Strict** en las cookies de sesion, que instruye al navegador a no incluir la cookie en peticiones originadas desde sitios externos, y con la verificacion de los encabezados **Origin** y **Referer** en el servidor. Frameworks modernos como Spring Security, Django o Laravel implementan la proteccion CSRF de forma automatica, lo que facilita su adopcion sin requerir implementacion manual por parte del desarrollador.